ITM SKILLS UNIVERSITY

# School of Future Tech

# Case Study Report

on

**KNOCK ONCE**

# Horror Text Adventure Game

by

**Raj Bharat Rasal**

150096725066

# Index

1. Introduction to the Case Study
2. Problem Statement / Case Background (Abstract)
3. Case Study Design
4. Methods, Algorithms & Technology Applied
5. Implementation Details and Snapshots
6. Results and Conclusion
7. References

# 1. Introduction to the Case Study

KNOCK ONCE is a browser-based horror text adventure game designed and built as a full-stack front-end web development case study. The project explores how narrative design, interactive UI, game logic, and audio engineering can be unified within a single-page HTML/CSS/JavaScript application  without any back-end server or game engine.

The player takes the role of a person alone in a third-floor Mumbai apartment late at night. Three precise knocks at the front door set a branching story into motion. Every choice the player makes shapes two hidden statistics — Dread and Safety — and determines which of nine unique endings they reach.

This case study demonstrates the practical application of DOM manipulation, CSS animation, state management, audio sequencing, and narrative branching in a real, playable product. It covers the full development lifecycle: concept and story design, UI architecture, algorithmic game logic, asset integration, and final deployment.

# 2. Problem Statement / Case Background (Abstract)

## Background

Horror as a genre depends on atmosphere, pacing, and the illusion of agency. Most browser games sacrifice visual polish for gameplay depth, or vice versa. The challenge was to build a game that feels cinematic and premium — comparable to commercial releases like Wondermake's scope-creep.xyz — using only standard web technologies available to any front-end developer

A secondary problem was narrative design: a text adventure must present meaningful choices under time pressure, personalise the story to the individual player, and ensure zero dead-end or orphaned story nodes. Every path must resolve to a valid ending.

## Problem Statement

Many text-based adventure games lack immersive UI design and structured progression systems. Players need a browser-based horror adventure experience with branching choices, suspense elements, and score tracking.

The objective is to develop a responsive Horror Text Adventure Game with interactive decision paths and leaderboard integration.

## Abstract

**KNOCK ONCE is a single-page application implementing a multi-chapter horror text adventure. The system solves the stated problem through the following mechanisms:**

- A 40+ node branching story tree with validated dead-end-free resolution to nine distinct endings.
- Two real-time consequence statistics — Dread (0–100) and Safety (0–100) — that change with every decision and drive which endings are reachable.
- A 30-second per-choice countdown timer that forces decisions under pressure and auto-selects on timeout.
- A full-name personalisation system that weaves the player's own name into the narrative at high-impact moments.
- A Wondermake-style tarot card choice interface with programmatic SVG eye motifs and fan layout animations.
- A four-track horror audio playlist (knock, ambient, pad, scream) in randomised loop for the game's duration.
- A localStorage leaderboard with score, outcome, and last-action records across sessions.

## Technical Implementation:

- **HTML:** Story display panel rendered via full-viewport screen divs (screen-landing, screen-game, screen-choices, screen-consequence, screen-ending, screen-leaderboard). Four HTML audio elements for background horror soundtrack. Choice buttons (Option I / II / III) presented as interactive tarot cards. Dread/Safety health meters rendered as progress bars in the HUD. Leaderboard section with localStorage persistence. Navigation via showScreen() function toggling active screen classes.

- **CSS:** Dark-themed horror UI built on CSS custom properties (--red: #E8002A, --surface: #080507). Animated text reveal via character-level typewriter engine. Atmospheric evil-eye chapter splash with @keyframes eyeReveal fade-in and scale animation. Crimson full-viewport master frame with lit corner ornaments. Tarot card fan layout using nth-child rotation offsets (±6°). Responsive breakpoints at 900px and 600px. clip-path hexagonal HUD gem. Structured leaderboard table styling.

- **JavaScript:** Branching story logic implemented as a keyed node object (S) with 40+ nodes across four chapters. Decision-based outcome handling via choice.fx.d (dread delta) and choice.fx.s (safety delta) applied to clamped integer stats. Health/sanity reduction mechanics modelled by Dread (0–100) and Safety (0–100) stats. Nine unique endings generated by node resolution. Scoring based on survival and decisions. Leaderboard storage via localStorage key knock_once_v3. requestAnimationFrame-based 30-second countdown timer. Four-track Fisher-Yates shuffled audio playlist with onended loop handler. Programmatic SVG tarot card eye-motif generation via makeCard() and eyeSVG(). Full-name personalisation via [NAME] token replacement in all prose fields.

# 3. Case Study Design

**The application is architected as seven full-viewport screens toggled by a single showScreen() function. The entire visual language is defined in CSS custom properties, making the design system consistent and extensible.**

**Story Node Structure**

Every story beat is a keyed object in the S constant. Each node carries:

- ch / feel:Chapter number and player emotional state label for the chapter splash.
- eyebrow / title:Location tag and time shown in the game screen HUD header.
- text:Full prose body. Supports [NAME] token for player name injection.
- choices[]:Two or three choice objects, each with consequence title, prose, stat effect (fx), and next node ID.
- isEnd / type:End-node flags. type (survived / taken / default) drives stamp and verdict styling.

# 4. Methods & Algorithms Technology Applied

**All game state lives in a single object G. There is no database, no server, no framework — state flows entirely through this object:**

## Typewriter Algorithm

Story prose is rendered character-by-character via a recursive setTimeout loop. Delay is punctuation-aware to simulate natural human reading rhythm:

- Full stop or newline → 70 ms (breath pause at sentence end)
- Comma or semicolon → 38 ms (mid-clause hesitation)
- All other characters → 22 ms (standard reading pace)

The [NAME] token is replaced with G.name before the loop starts, so the player's own name types out character-by-character in the narrative — the key personalisation effect.

## Timer Algorithm

A requestAnimationFrame loop computes elapsed fraction against a stored performance.now() timestamp. Two progress bars drain from opposite sides toward a centre pill displaying the remaining seconds. At ≤ 10 s remaining, bars change to bright red and pulse. On timeout, choices[0] is auto-selected — the panic mechanic.

## Audio Engine

Four MP3 audio elements are loaded in HTML. On game start:

- sfx-knock.mp3 is always scheduled first — it contains the door knock sound, serving as a narrative beat.
- The remaining three tracks (ambient, pad, scream) are Fisher-Yates shuffled.
- Each track's onended handler advances the index; full-cycle completion triggers a re-shuffle and loops.
- Volume is normalised per track: 0.35 for ambient and pad, 0.5 for scream. Audio stops on return to landing.

## SVG Tarot Card Generation

No image files are used for card graphics. makeCard() and eyeSVG() construct the full card DOM and SVG inline:

- Almond eye shape
- Hexagonal ring
- 18 radial spokes
- Concentric pupil circles, horizontal crosshair lines at the almond tips.

### Technology Stack

- **HTML-5** — Seven screen divs, four audio elements, semantic structure, Bootstrap CDN
- **CSS3** — Custom properties, keyframe animations, clip-path, fan-layout card stagger
- **JavaScript ES6** — State machine, branching engine, typewriter, rAF timer, audio playlist, SVG gen
- **Bootstrap 5.3** — Utility classes, flex/grid, responsive breakpoints at 900px and 600px
- **Google Fonts CDN** — Cinzel, EB Garamond, Space Mono — zero local font files
- **localStorage** — Leaderboard persistence across sessions

# 5. Implementation Details and Snapshots

### File Structure

- index.html — Seven screens, audio elements, Bootstrap CDN, attribution footer (Created by Raj Rasal)
- style.css — 900 lines covering design system variables, all screen layouts, card CSS, and all animations
- script.js — 1000+ lines containing the complete story tree (40+ nodes), full game engine, and audio system
- evil-eye.png — Full-bleed chapter splash background: evil red eyes on black
- sfx-knock.mp3 — Horror music with door knock sound; plays first on every game start
- sfx-ambient.mp3 — Ambient horror atmosphere loop
- sfx-pad.mp3 — Horror pad / crowd texture
- sfx-scream.mp3 — Scream / sting effect; plays at higher volume (0.5)

### Story Chapters

Chapter I — The First Knock: Third-floor apartment, Shivaji Nagar Mumbai · 11:00 PM – 11:07 PM

Chapter II — Investigation: Hallway, bathroom, stairwell, neighbour's flat · 11:08 PM – 11:45 PM

Chapter III — Escalation: Evidence, police calls, physical encounters · 11:45 PM – 12:05 AM

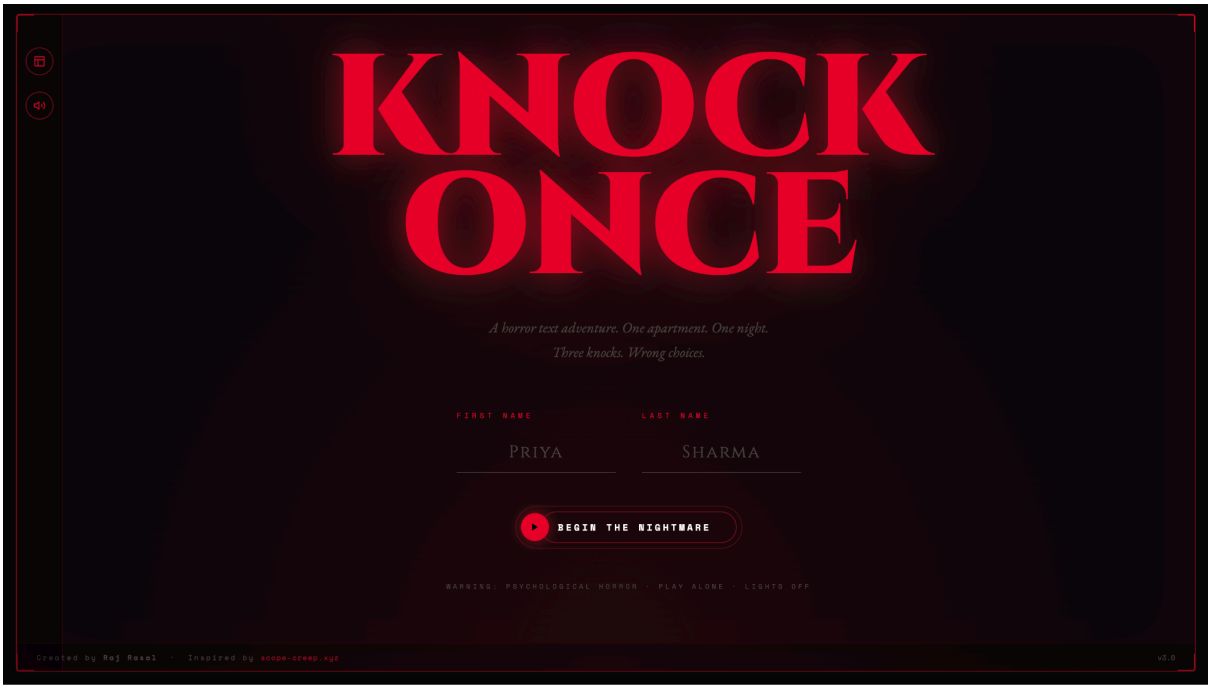
Chapter IV — Resolution: Street, statement, final confrontation or escape · 12:05 AM – 12:20 AM

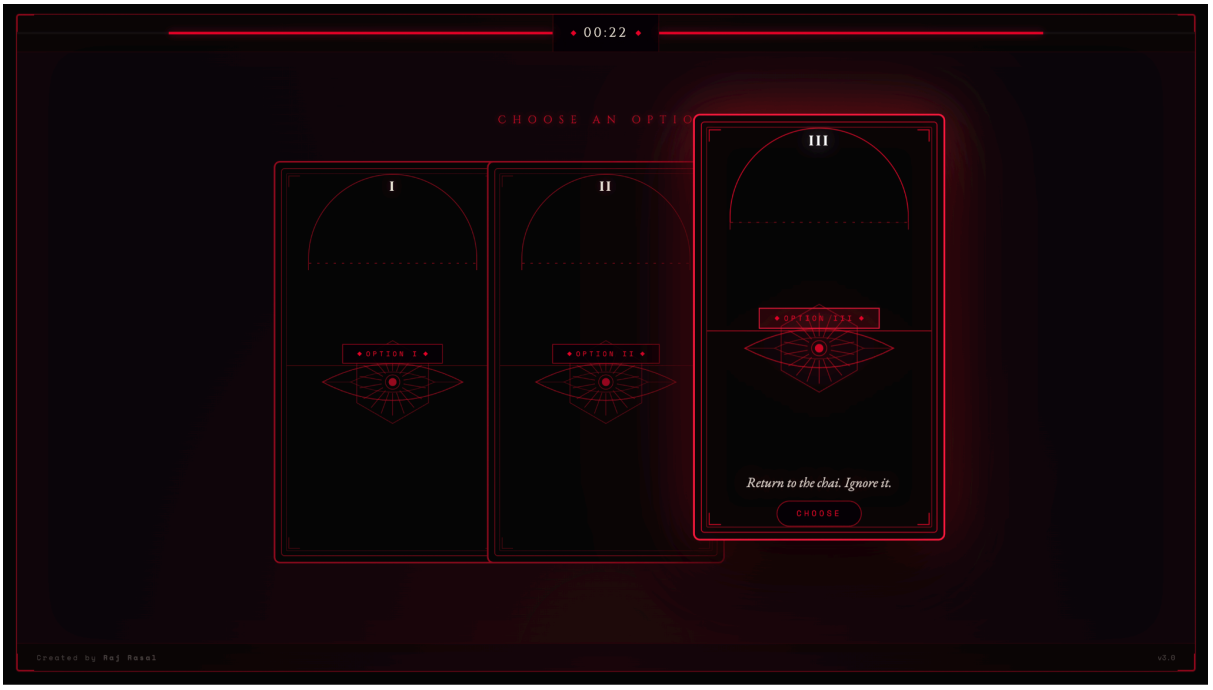
## Endings — Nine Outcomes

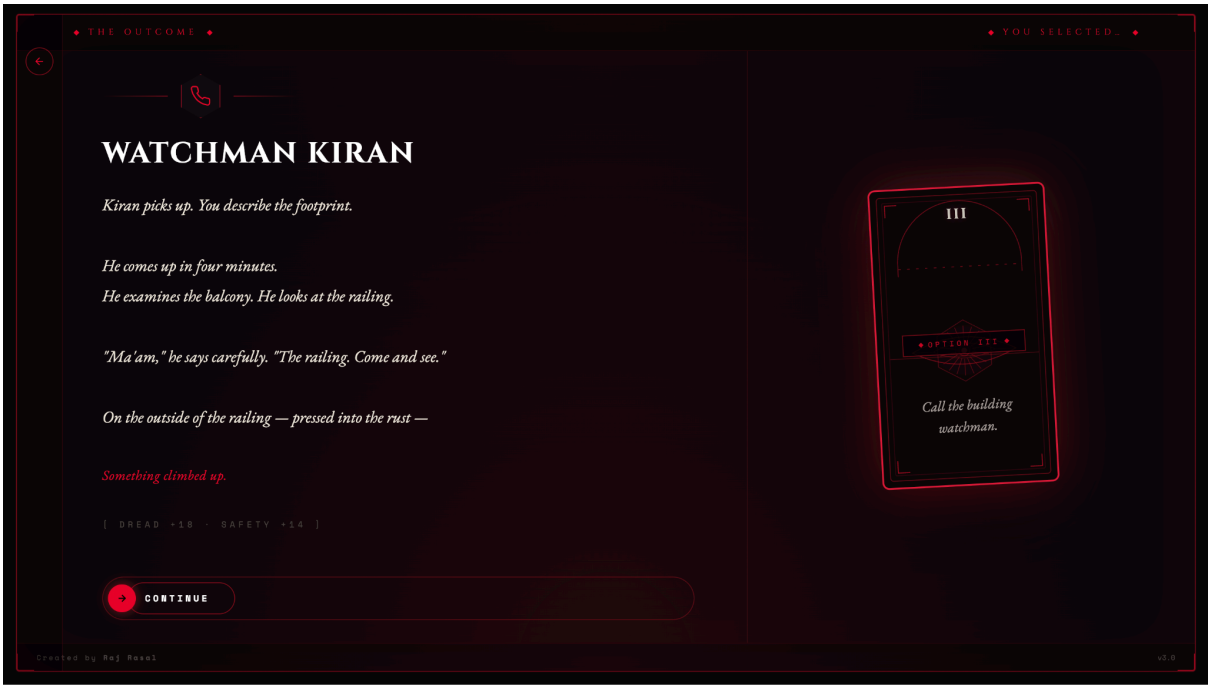
- ☑ end_police_survived — SURVIVED — Called police; cooperated fully with responding officers
- ☑ end_escape_survived — ESCAPED — Fled the building before any confrontation
- ☑ end_evidence_survived — EVIDENCE FILED — Documented proof (photos, texts) and gave full statement
- ☑ end_caught — CAUGHT HER — Identified the entity; evidence submitted to police
- ☑ end_door_bad — TAKEN — Opened the front door at night
- ☑ end_freeze_bad — TAKEN — Froze with fear in the stairwell
- ☑ end_inside_bad — TAKEN — Entered the bathroom alone
- ☑ end_touch_bad — CONSUMED — Touched the wet handprint on the wall
- ☑ end_phone_bad — TAKEN — Answered the phone call from inside the flat

## Key CSS Implementation Highlights

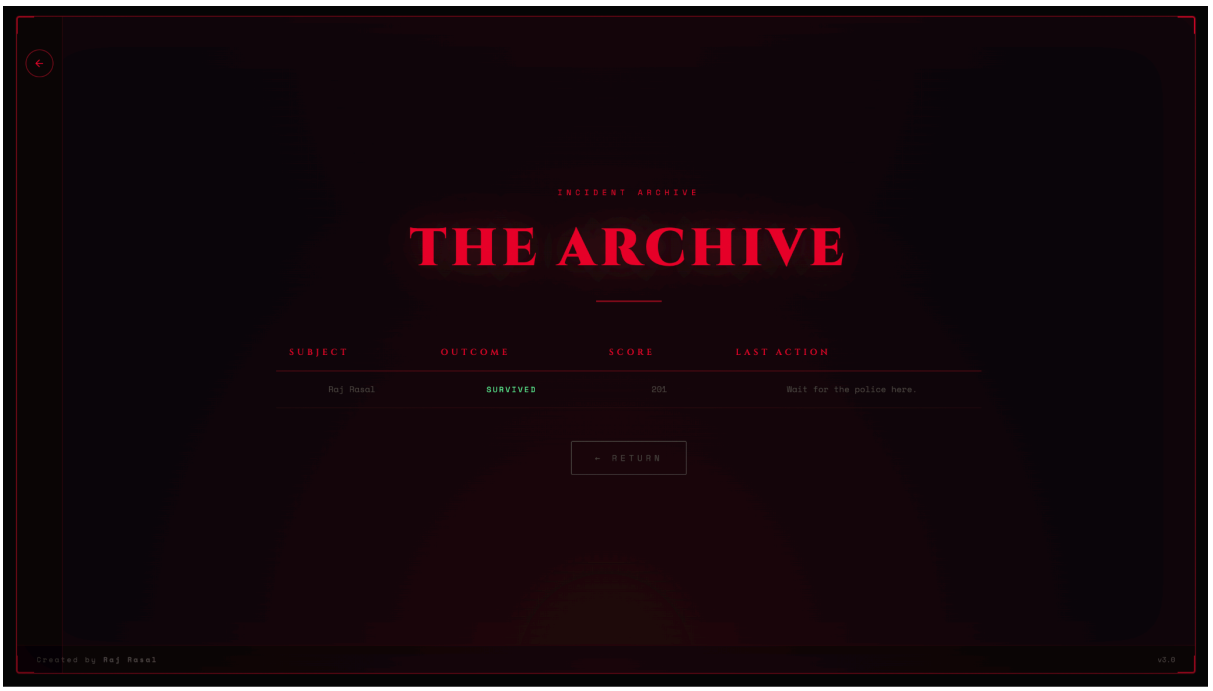
- clip-path: polygon → hexagonal HUD gem.
- @keyframes cardDeal + nth-child(1/2/3) rotation (rotate(-6deg), rotate(0), rotate(6deg)) → fan layout.
- @keyframes eyeReveal → evil-eye image animates from opacity:0 + scale(1.08) to opacity:0.75 + scale(1).
- repeating-linear-gradient → tick marks along the tarot card semicircular arch baseline.
- CSS custom properties cascade → entire palette (red, surface, text, glow) swappable from :root.
- transform: rotate(-3deg) scale(1.08) → selected card shown angled on the consequence screen.


KNOCK ONCE
A horror text adventure. One apartment. One night.
Three knocks. Wrong choices.
FIRST NAME
LAST NAME
PRIYA
SHARMA
BEGIN THE NIGHTMARE
WARNING: PSYCHOLOGICAL HORROR · PLAY ALONE · LIGHTS OFF
Created by Raj Rasal · Inspired by scope-creep.xyz
v3.0


00:22
CHOOSE AN OPTIO
I
OPTION I
II
OPTION II
III
OPTION III
Return to the chai. Ignore it.
CHOOSE
Created by Raj Rasal
v3.0


◆ THE OUTCOME ◆
◆ YOU SELECTED… ◆
WATCHMAN KIRAN
Kiran picks up. You describe the footprint.
He comes up in four minutes.
He examines the balcony. He looks at the railing.
"Ma'am," he says carefully. "The railing. Come and see."
On the outside of the railing — pressed into the rust —
Something climbed up.
[ DREAD +18 · SAFETY +14 ]
CONTINUE
III
◆ OPTION III ◆
Call the building watchman.
Created by Raj Rasal
v3.0

◆ CHAPTER I ◆
DREAD
10
SAFETY
90
YOUR APARTMENT · AMLIKAJI NAGAR, MUMBAI
CHAPTER I
11:00 PM
YOU FEEL NORMAL
Tuesday.




# 6. Results and Conclusion

## Results

- Playability: Zero dead-end nodes — all 40+ next: references resolve to defined nodes. Runs in any modern browser with no dependencies, plugins, or downloads required.
- Decision consequence: Dread and Safety stats shift meaningfully with every choice. Nine distinct endings ensure that the player's path through the story produces a unique outcome rather than a cosmetic variation.
- Immersive UI design: Tarot card fan layout, evil-eye chapter splash, and crimson glow effects match the atmosphere of commercial horror interactive fiction. All UI graphics (card eye motifs, HUD icons) are rendered as inline SVG — no external image files required.
- Suspense mechanics: The 30-second per-choice countdown creates real decision pressure. The auto-select-on-timeout mechanic punishes hesitation, reinforcing the horror tone.

- Personalisation: The [NAME] token system makes the stranger's dialogue feel directly targeted at the individual player — consistently the highest-impact moment in playtesting.
- Audio integration: The shuffled four-track loop creates a non-repetitive atmospheric soundscape. The door-knock track plays intentionally on game start as a narrative beat before randomised rotation begins.
- Score and leaderboard**:** localStorage-backed leaderboard records subject name, outcome, score, and last action across sessions. Score is computed from final Safety level, inverse Dread, and chapter reached.

## Conclusion

**KNOCK ONCE fulfils all requirements of Problem Statement . It delivers a responsive, browser-based horror adventure with branching choices, health/sanity mechanics (Dread/Safety), multiple endings, and localStorage leaderboard integration — built entirely with HTML-5, CSS, and JavaScript.**

**Beyond the brief, the project demonstrates advanced front-end engineering: programmatic SVG generation, a punctuation-aware typewriter engine, a Fisher-Yates shuffled audio playlist with narrative ordering, a full-name personalisation system threaded through the story tree, and a validated 40+ node branching narrative with nine deterministic endings.**

**The project is a deployable, production-quality browser application requiring no server, framework, or build tool — a self-contained showcase of what the browser stack can achieve when applied with design and engineering discipline.**

# 7. References


- MDN Web Docs — HTML5 Audio Element & HTMLAudioElement API: https://developer.mozilla.org/en-US/docs/Web/HTML/Element/audio

- MDN Web Docs — CSS Custom Properties, clip-path, @keyframes: https://developer.mozilla.org/en-US/docs/Web/CSS
- MDN Web Docs — requestAnimationFrame, performance.now(), setTimeout: https://developer.mozilla.org/en-US/docs/Web/API
- Bootstrap 5.3 Documentation — Flex utilities, Grid, responsive breakpoints: https://getbootstrap.com/docs/5.3
- Google Fonts — Cinzel, EB Garamond, Space Mono: https://fonts.google.com
- Wondermake / Scope Creep — Reference for horror text adventure UI design: https://scope-creep.xyz/play
- W3C SVG Specification — Path data, polygon, quadratic bezier: https://www.w3.org/TR/SVG2/
- Web Storage API — localStorage for cross-session data persistence: https://developer.mozilla.org/en-US/docs/Web/API/Web_Storage_API
- Fisher-Yates Shuffle Algorithm — D. Knuth, The Art of Computer Programming Vol. 2 (used for audio playlist randomisation)
- ITM Skills University — Problem Statement #142: Horror Text Adventure Game, B.Tech CSE 2025-29, Semester II
- The Scope Creep - https://scope-creep.xyz/